**UCS 503 - Software Engineering**

# SEOS

Software Engineering Operating System: An AI-Powered Workspace for Learning, Designing and Engineering Software


**Submitted by:**

1024160035 Tanveer Chawla
1024160057 Divyanshi Dhawan
1024160064 Harshit Garg

Group: L2[3P12] BE Third Year, CSE

**Submitted to:**

Sandeep Kaur


Mid-Semester Evaluation


**Computer Science and Engineering Department**
Thapar Institute of Engineering and Technology, Patiala

# Contents

**1 Introduction** **3**
1.1 Background and Context . . . 3
1.1.1 Problem Statement . . . 3
1.2 Project Objectives . . . 4

**2 Methodology and Design** **5**
2.1 System Architecture . . . 5
2.1.1 High-Level Design . . . 6
2.1.2 Technical Stack . . . 6
2.2 Implementation Details . . . 6
2.2.1 Module 1: Core Functionality . . . 6
2.2.2 Module 2: Secondary Features . . . 10

**3 Results and Evaluation** **15**
3.1 Testing Methodology . . . 15
3.2 Prototype Walkthrough . . . 15
3.2.1 Project Overview . . . 16
3.2.2 Requirements Engineering . . . 17
3.2.3 System Modeling . . . 18
3.2.4 Architecture Recommendation . . . 19
3.2.5 API Designer . . . 20
3.2.6 Documentation . . . 21
3.2.7 Learning Dashboard . . . 22
3.3 Performance Metrics . . . 23
3.4 Analysis . . . 23

**4 Conclusion** **25**
4.1 Summary of Work . . . 25
4.2 Key Findings . . . 25
4.3 Future Work and Recommendations . . . 25
4.4 Final Remarks . . . 26

**Bibliography** **27**

# Chapter 1
# Introduction

## 1.1 Background and Context

Software engineering is usually taught as a sequence of separate topics: requirements elicitation, user stories, UML and entity–relationship (ER) modeling, architecture, API design, testing and documentation [Som15, PM19]. Each topic is assessed with its own artefact, and students typically produce those artefacts in different tools — requirements in a word processor, class diagrams in a drawing tool, schemas in a database designer, API contracts in Swagger, and reports in Markdown or Word.

In real projects, however, these artefacts are tightly coupled. Adding a single field to an entity should change the database schema, the request and response schemas of the API, and the relevant sections of the Software Requirements Specification (SRS) and Software Design Document (SDD). When the artefacts live in unrelated tools, keeping them consistent is a manual, error-prone task and the *reasoning* that connects one artefact to the next is rarely made explicit.

*SEOS* (Software Engineering Operating System) is an AI-powered workspace that takes a natural-language software idea and guides it through the complete engineering pipeline — from requirements to documented, deployable design — while explaining why each engineering decision was made. The platform is intentionally domain-agnostic: the same workflow applies to a food-delivery platform, a hospital system, a college application or a banking system. In this report the running case study is *FoodDash*, an on-demand food delivery platform generated end-to-end inside the current SEOS prototype.

### 1.1.1 Problem Statement

Students and small teams building software projects face three distinct challenges:

1. **Artefact Fragmentation**: Requirements, diagrams, schemas, APIs and documents are produced in separate tools with no single source of truth. A change in one artefact must be manually propagated to every downstream artefact, and inconsistencies accumulate silently.

2. **Black-Box AI Generation**: General-purpose AI assistants can produce UML diagrams or API specifications on request, but each output is an isolated text blob. They neither maintain relationships between artefacts nor justify their choices with established software engineering principles, so learners copy outputs without understanding them.

3. **Missing Learning Feedback**: Existing tools focus on producing artefacts, not on building understanding. Learners receive no signal about which concepts they have encountered, which stages they have completed, or why a particular architecture or normal form was chosen.

## 1.2 Project Objectives

The core objectives of SEOS are formulated under the SMART (Specific, Measurable, Achievable, Relevant, Time-bound) framework for a three-person team over a six-month timeline:

1. **Objective 1: Connected Engineering Model**: Implement a single relational *Project Model* in which requirements, actors, user stories, entities, relationships, architecture, API endpoints and documents are stored as linked objects with explicit trace links, so that a downstream artefact can always be traced to its origin.

2. **Objective 2: Validated AI Generation Pipeline**: Build an end-to-end pipeline (Idea $\rightarrow$ Requirements $\rightarrow$ User Stories $\rightarrow$ UML/ER $\rightarrow$ Architecture $\rightarrow$ OpenAPI $\rightarrow$ SRS/SDD/README) in which every LLM response is returned as structured JSON, validated against a schema, and approved by the user before it is persisted. Generated OpenAPI documents must be importable into Swagger or Postman without manual correction.

3. **Objective 3: Grounded Explanations and Learning Analytics**: Ground engineering explanations in a curated software engineering knowledge base using Retrieval-Augmented Generation (RAG) [Lew20] over PostgreSQL with `pgvector`, and track learner progress through a Learning Dashboard. At least five external users should complete the full workflow by Month 6.

# Chapter 2
# Methodology and Design

## 2.1 System Architecture

SEOS follows a layered client–server architecture partitioned into four layers: a React/TypeScript presentation layer, a FastAPI service layer, an AI and RAG orchestration layer, and a PostgreSQL persistence layer extended with `pgvector` for similarity search. All frontend–backend communication uses REST with JSON payloads; the LLM provider is treated as an external service accessed only through a gateway in the AI layer. Figure 2.1 shows the component view.

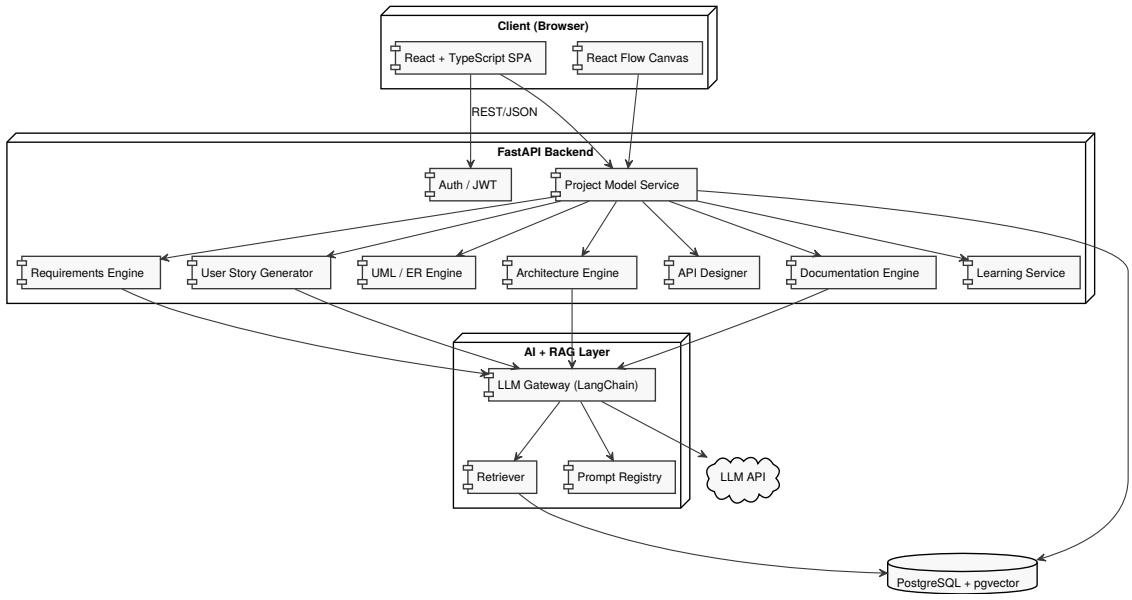



Figure 2.1: SEOS Component Architecture

The `Project Model Service` is the hub of the backend. Every generation engine reads its inputs from, and writes its outputs back to, the Project Model, so diagrams, API specifications and documents become *views* of the model rather than independent

outputs. This is the central design decision that enables change propagation and traceability.

### 2.1.1 High-Level Design

The interaction between system actors and functional boundaries is modeled in the Use Case Diagram in Figure 2.2.

The diagram shows the abstract `User` actor specialised into `Student / Learner` and `Mentor`, alongside the external `LLM Provider` and the internal `RAG Retriever` subsystem. It highlights the mandatory `«include»` relationships — project creation always includes clarification, and both architecture recommendation and the AI assistant include grounded retrieval — and the optional `«extend»` branch by which user-story generation extends requirement approval.

### 2.1.2 Technical Stack

The platform is built on open-source frameworks and a hosted LLM API:

- **Framework/Language:** React with TypeScript [Met24] and Tailwind CSS on the frontend; React Flow [RF24] for the interactive UML/ER canvas; Python with FastAPI [Fas24] on the backend.
- **Database:** PostgreSQL for relational project data, extended with `pgvector` [PGV24] to store embeddings for RAG retrieval in the same database.
- **AI:** An LLM API (Gemini Flash in the current prototype) orchestrated through LangChain [LC24], with versioned prompts and Pydantic models for structured output validation.
- **Security:** JSON Web Tokens (JWT) [JBS15] for stateless authentication, salted password hashing, and project-level authorization.
- **Tooling:** Docker for reproducible builds, GitHub Actions for CI, and Vercel (frontend) plus Render (backend) for deployment.

## 2.2 Implementation Details

### 2.2.1 Module 1: Core Functionality

The core domain is structured around the relational schema illustrated in the Class Diagram in Figure 2.3.

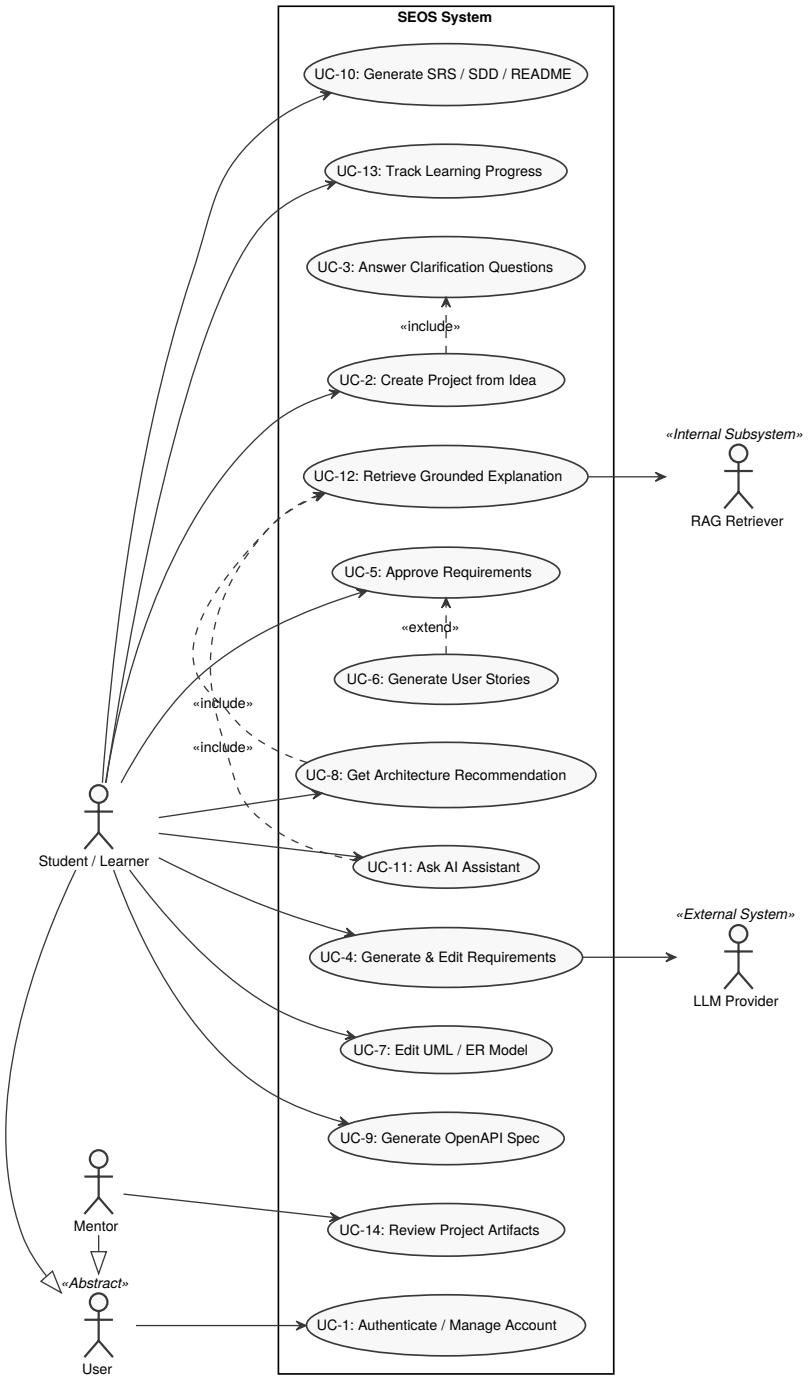



Figure 2.2: SEOS Use Case Diagram

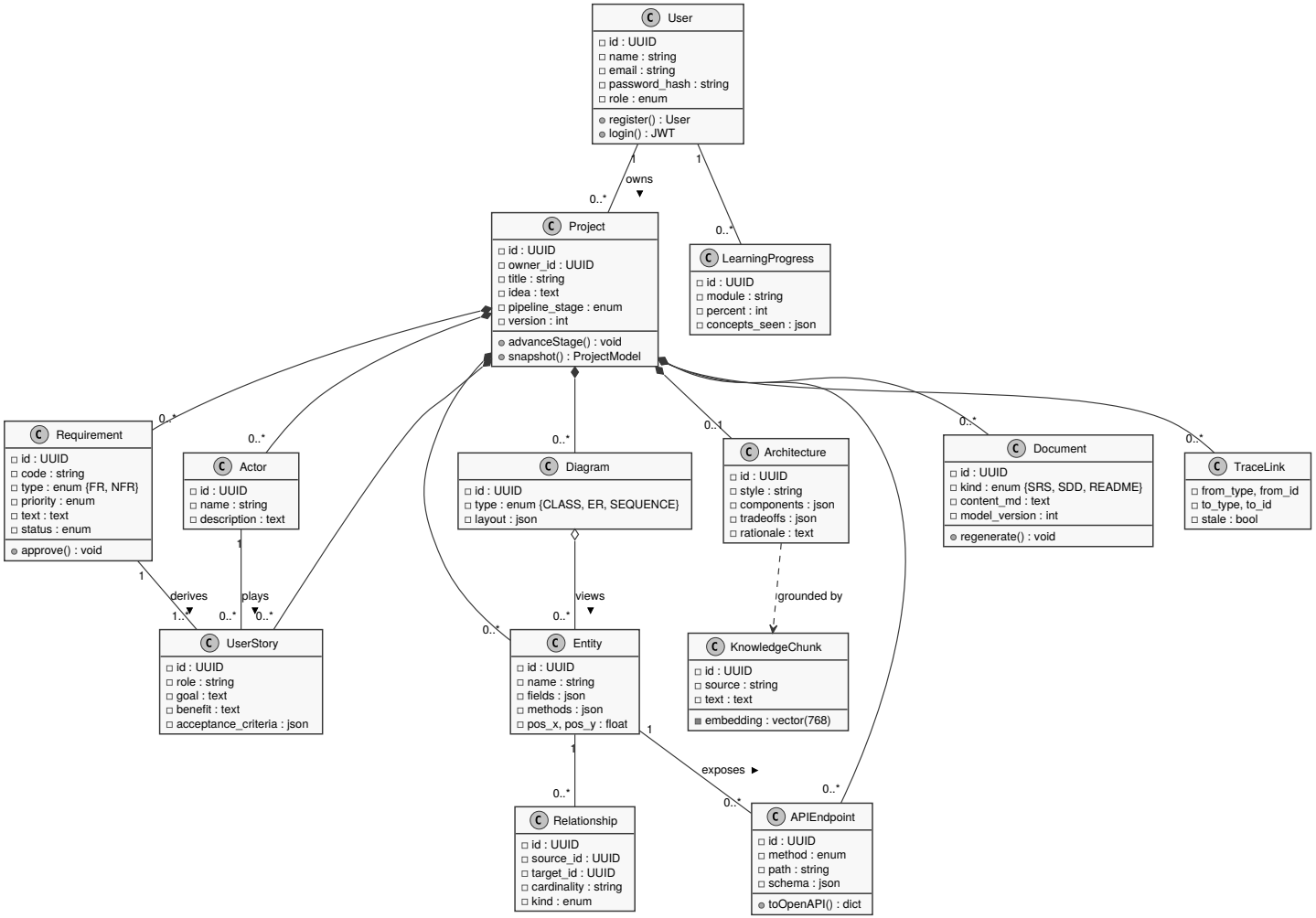



Figure 2.3: SEOS Core Domain Class Diagram (Project Model)

Figure 2.3 models the 15 core domain entities of the Project Model. `Project` is the aggregate root and composes every engineering artefact (`Requirement`, `Actor`, `UserStory`, `Entity`, `Diagram`, `Architecture`, `APIEndpoint` and `Document`). `Diagram` *aggregates* rather than composes `Entity`, reflecting that a diagram is only a view: deleting a diagram must never delete the model objects it displays. The generic `TraceLink` table records directed dependencies between any two artefacts and carries a `stale` flag used for change propagation.

### 1. AI Requirements Engineer

Rather than generating a long document from a vague prompt, the requirements engine first identifies ambiguity and asks targeted clarification questions about actors, goals, system boundaries, constraints and assumptions. The backend sends the idea plus conversation context to the LLM, which must return JSON conforming to a predefined `RequirementSet` schema. Each requirement carries a type (Functional or Non-Functional), a priority (Critical, High, Medium) and a status (Draft or Approved). Invalid responses are sent back to the LLM with the validation errors in a bounded repair loop (at most two retries) before the user is asked to clarify. For the FoodDash case study the engine produced six requirements (four functional, two non-functional), two of which have been approved (Figure 3.2).

The dynamic interaction between the student, frontend, backend, LLM gateway, schema validator and database is detailed in the Sequence Diagram in Figure 2.4. It traces three phases: idea submission with the validation/repair loop, transactional persistence of draft requirements together with their trace links, and the approval step that advances the project's `pipeline_stage` to user stories.

### 2. User Story Generator

Approved requirements are transformed into user stories of the form *"As a <role>, I want <goal> so that <benefit>"*, each with acceptance criteria. Every story stores a trace link to its source requirement, so the user can always see where a story originated. In FoodDash, the four core requirements were converted into four user stories with acceptance criteria.

### 3. Interactive UML & ER Designer

Entities, attributes and relationships are extracted from approved requirements and stories into the Project Model and rendered on a React Flow canvas (Figure 3.3). The diagram is not a static image: each node maps to a model object (*Diagram Node* $\leftrightarrow$ *Model Object* $\leftrightarrow$ *Database*). Users can open an entity and edit its fields, data types, primary/foreign-key flags and methods (Figure 3.4). The FoodDash ER model contains six entities — `User`, `Order`, `Restaurant`, `Payment`, `Delivery` and `MenuItem`.

### 2.2.2 Module 2: Secondary Features

#### 1. AI Architecture Designer

Once requirements and models exist, the architecture engine analyses scale, functional boundaries, coupling and operational complexity, and recommends an architectural style (layered, modular monolith or microservices) together with its trade-offs. The emphasis is on reasoning rather than on producing a diagram. For FoodDash — which needs real-time updates across three consumers (customer app, restaurant dashboard, driver app) and independent scaling of order processing and delivery tracking — SEOS recommended an *Event-Driven Microservices* architecture with Apache Kafka as the message backbone, listing eight components and explicit pros and cons (Figure 3.5). The recommendation is labelled *AI-Generated · Grounded via RAG*.

#### 2. RAG Explanation Layer

SEOS grounds its explanations in a curated knowledge base covering requirements engineering, UML, design principles, architecture, database design, API design, testing and documentation. Reference documents are chunked, embedded and stored in `pgvector`; at query time the top-$k$ chunks are retrieved by cosine similarity and passed to the LLM together with the relevant slice of the Project Model (Figure 2.5). The similarity between query embedding $\mathbf{q}$ and chunk embedding $\mathbf{d}_i$ is

$$\mathrm{sim}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \, \|\mathbf{d}_i\|}, \qquad \mathcal{C} = \underset{i}{\text{top-k}} \ \mathrm{sim}(\mathbf{q}, \mathbf{d}_i) \tag{2.1}$$

where $\mathcal{C}$ is the retrieved context set (initially $k = 5$).

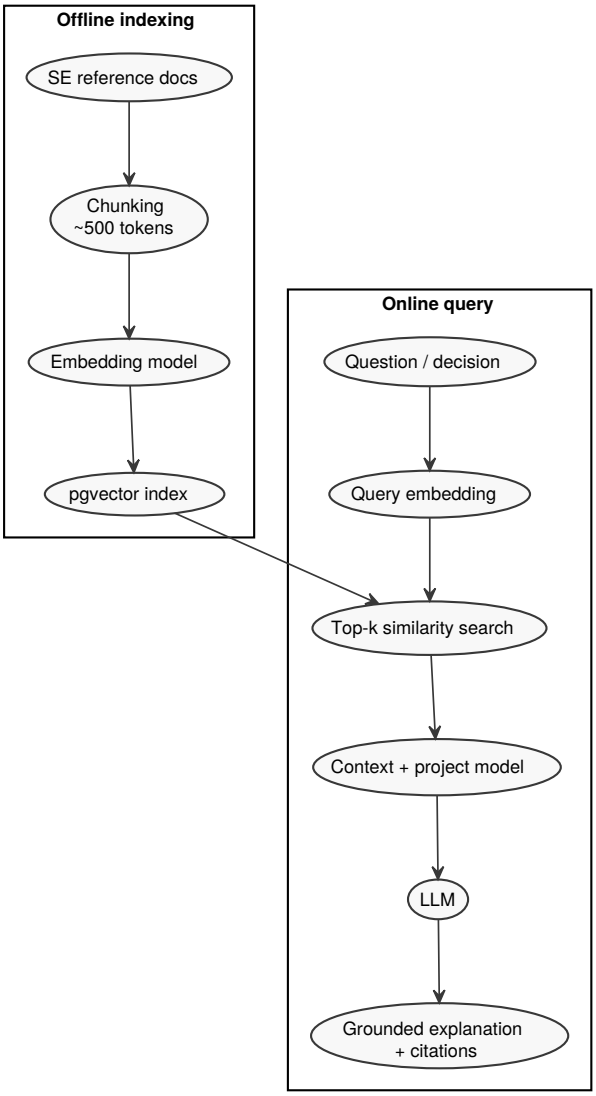



Figure 2.5: RAG Indexing and Query Pipeline

### 3. API Designer and Living Documentation

REST endpoints and schemas are derived from entities, relationships and user stories and serialised as an OpenAPI 3.1 document [OAS21] (Figure 3.6). The documentation engine then generates an SRS (following the structure of IEEE 830 [IEEE98]), an SDD and a README directly from the current model. Because each document stores the `model_version` it was generated from, SEOS can detect when a document is out of date and regenerate it — the idea of *living documentation* (Figure 3.7).

### 4. Change Propagation and Learning Dashboard

When a model object changes, the Project Model Service walks the trace links downstream and marks dependent artefacts as stale. For a project with artefact set $A$, the set of artefacts affected by a change to object $x$ is the transitive closure

$$\text{Stale}(x) = \{\, a \in A \mid x \rightarrow^{+} a \,\} \tag{2.2}$$

over the directed trace-link graph. The Learning Dashboard tracks completed pipeline stages, module progress and concepts encountered, and links them through a knowledge graph (e.g. SOLID, DDD, CQRS, Event Sourcing) (Figure 3.8). Stage progress for a module with $n$ checkpoints of which $c$ are complete is $P = (c/n) \times 100$.

The operational flow of a model edit is modeled in the Activity Diagram in Figure 2.6. It spans three swimlanes (`Student`, `SEOS Frontend`, `Project Model Service`) and details schema validation of the patch, the transactional update and version increment, the parallel fork that marks API endpoints and document sections as stale and re-runs consistency checks, and the user's choice to regenerate immediately or later.

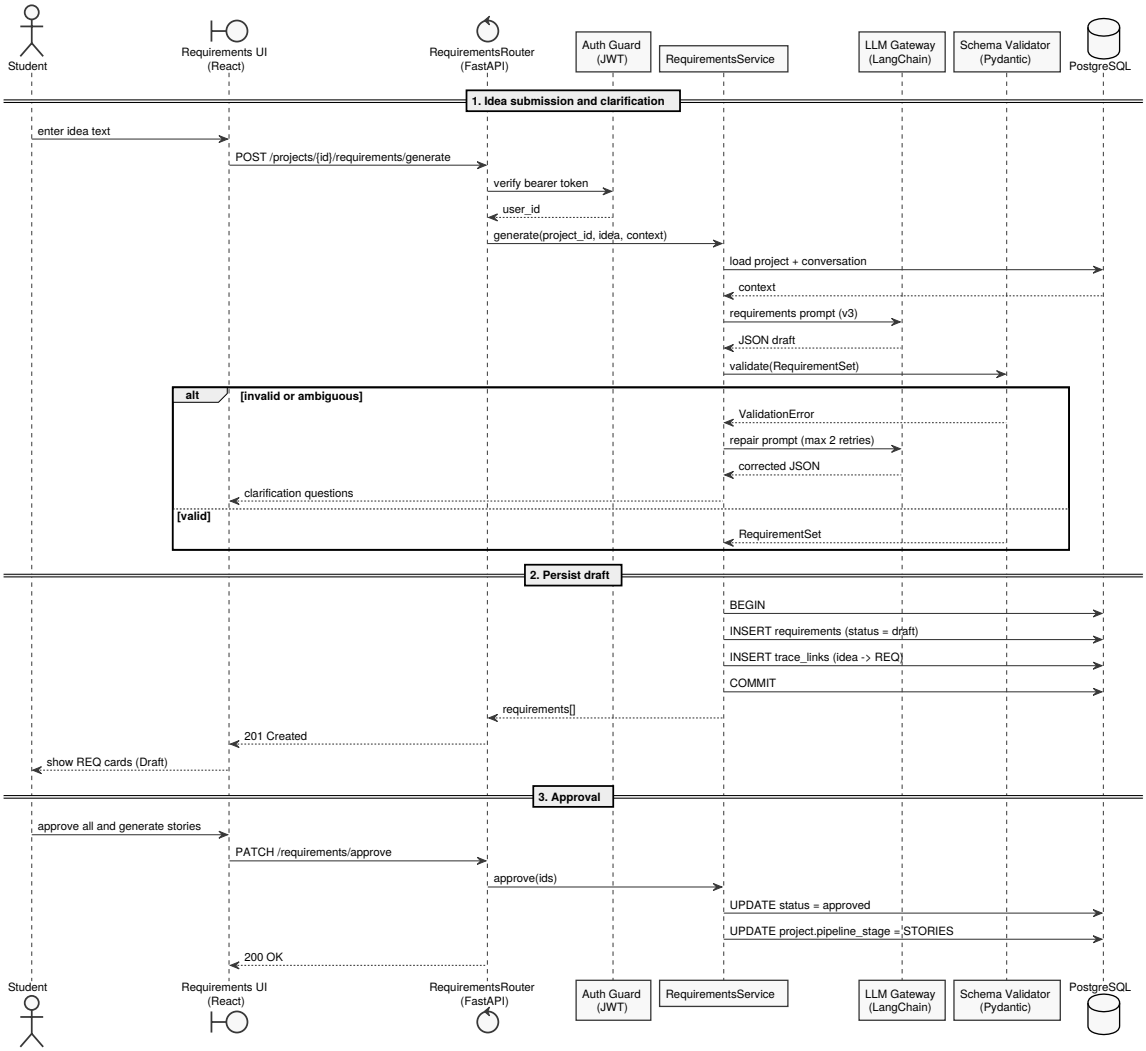



Figure 2.4: Sequence Diagram of Requirement Generation, Validation and Approval

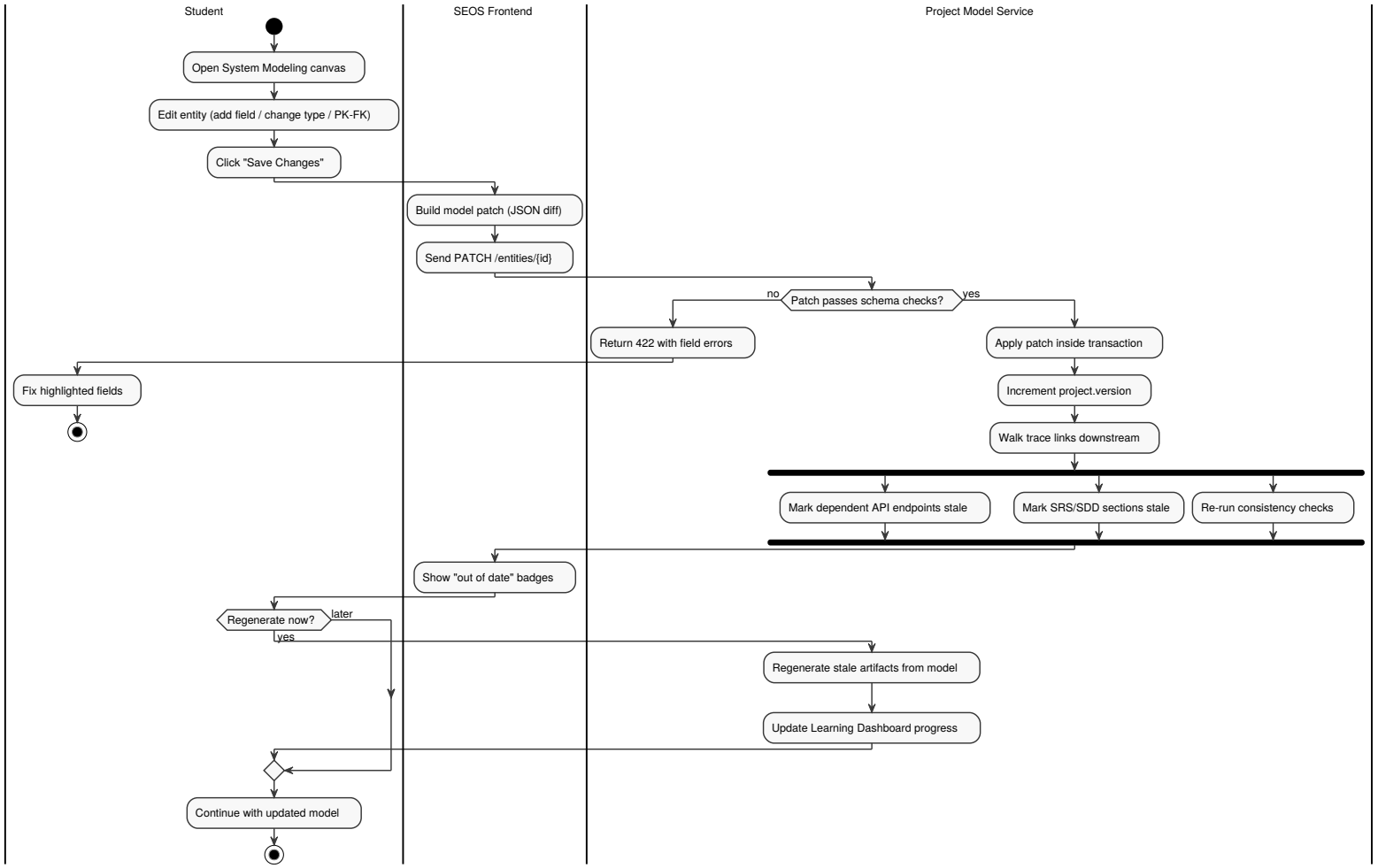



Figure 2.6: Activity Diagram for Model Editing and Change Propagation

# Chapter 3

# Results and Evaluation

## 3.1 Testing Methodology

The validation strategy combines automated testing of the backend with structured evaluation of AI outputs and users:

- **Unit Testing:** Backend services, JSON parsers, Pydantic schema validators, model transformations and utility logic using `pytest`.
- **Integration Testing:** Connected stages such as Requirements → User Stories, Model → API and Model → Documentation, exercised through FastAPI's test client.
- **End-to-End Testing:** The complete Idea → . . . → Documentation pipeline on reference projects (FoodDash, a hospital system and a college events portal).
- **AI Evaluation:** A fixed set of prompts scored for correctness, completeness, consistency, formatting, grounding and hallucination; OpenAPI output is checked with an OpenAPI validator.
- **Usability Testing:** At least five external users complete the full workflow and report whether explanations improved their understanding.

## 3.2 Prototype Walkthrough

The current prototype was exercised end-to-end on the FoodDash case study. The screens below follow the order of the engineering pipeline, from the project overview to the Learning Dashboard.

### 3.2.1 Project Overview

The overview dashboard summarises artefact counts (six requirements, four user stories, one ER model and three documents), the pipeline status — with Requirements, User Stories and Modeling complete and Architecture in progress — the detected technology stack, and a timeline of recent AI activity (Figure 3.1). The AI Assistant panel on the right is available on every screen.

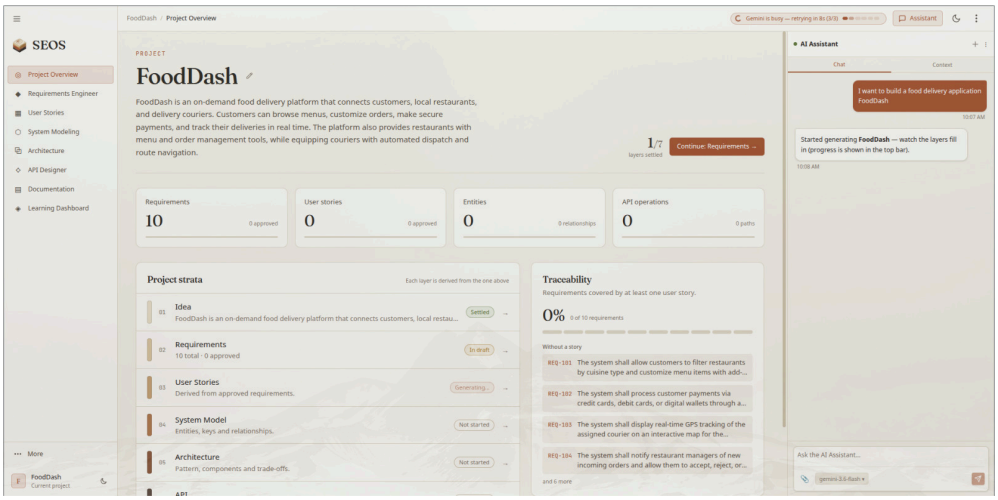



Figure 3.1: Project Overview dashboard for FoodDash (pipeline at the Architecture stage)

### 3.2.2 Requirements Engineering

Each generated requirement is shown as a card with its type, priority and approval status, and can be edited, approved individually, or approved in bulk to trigger user-story generation (Figure 3.2).

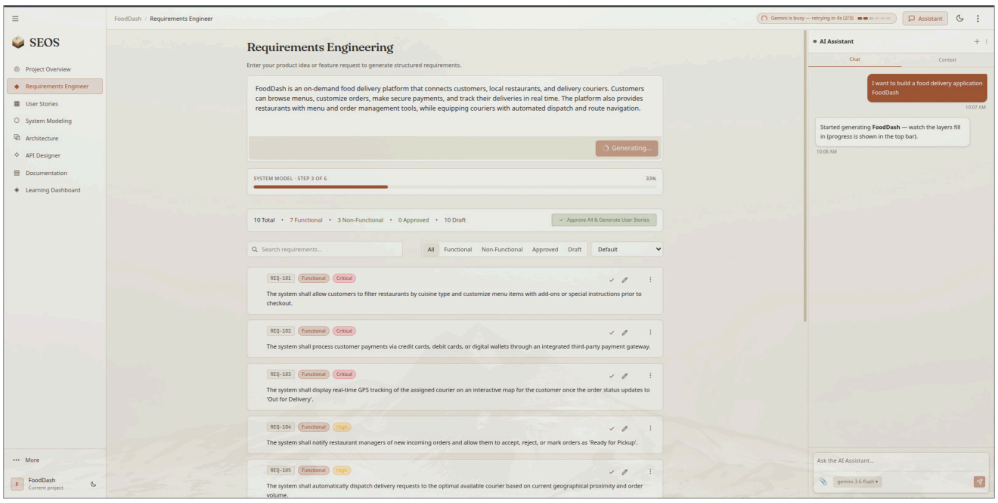



Figure 3.2: Requirements Engineer screen with functional and non-functional requirements

### 3.2.3 System Modeling

The ER model is rendered on an interactive canvas where entities can be moved, connected and deleted (Figure 3.3). Opening an entity shows the field editor with data types, PK/FK flags and methods (Figure 3.4).

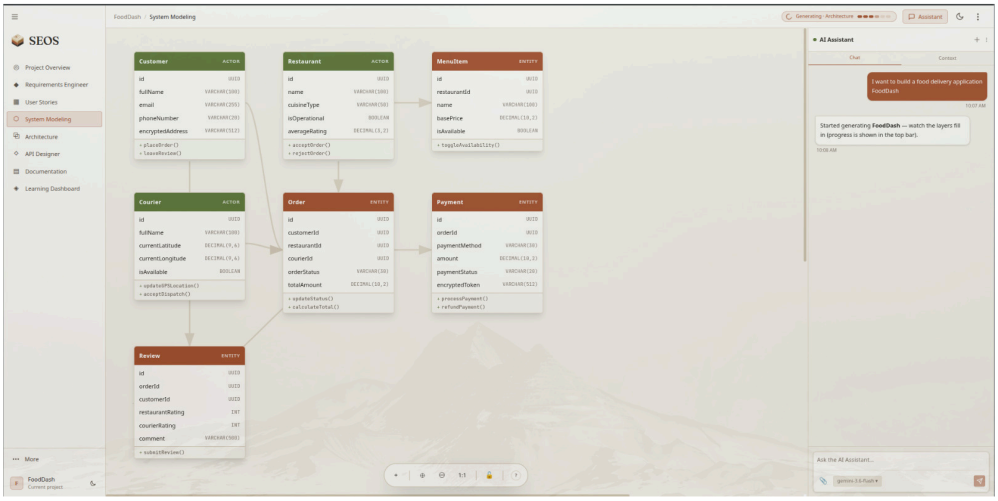



Figure 3.3: Interactive ER canvas generated for FoodDash (six entities)

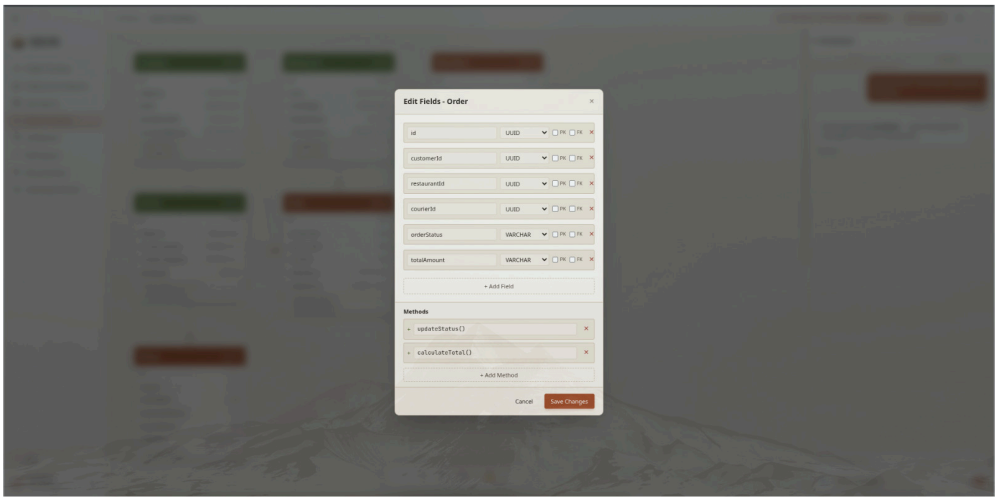



Figure 3.4: Entity editor for Payment: fields, types, PK/FK flags and methods

### 3.2.4 Architecture Recommendation

The architecture screen presents the recommended style, a component table with technology and purpose for each service, explicit pros and cons, and a layered architecture diagram (Figure 3.5).

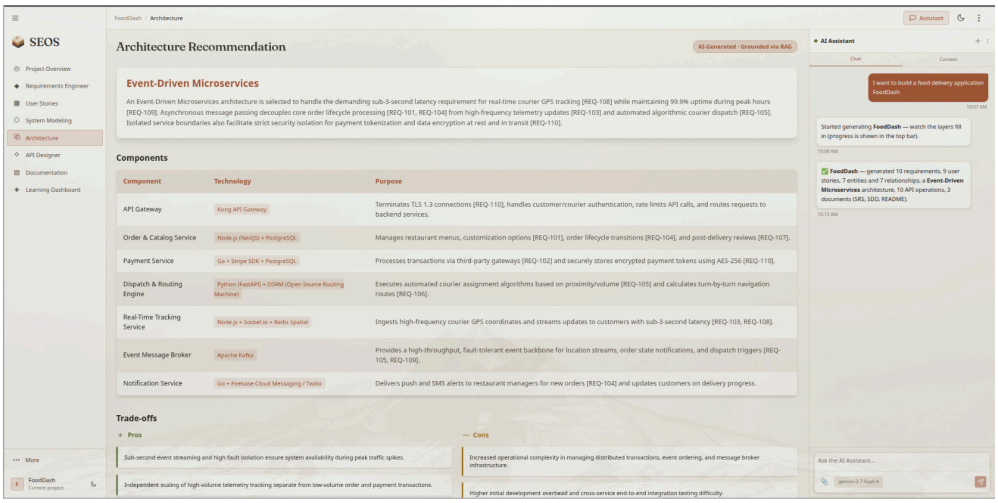



Figure 3.5: Architecture recommendation: Event-Driven Microservices with components and trade-offs

### 3.2.5 API Designer

The API Designer shows the generated `openapi.yaml` beside a list of endpoints colour-coded by HTTP method (Figure 3.6).

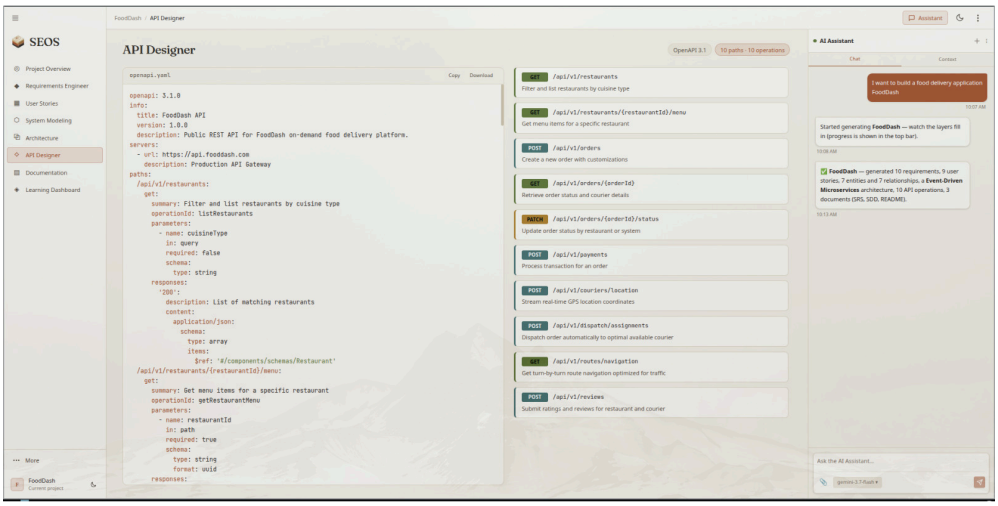



Figure 3.6: API Designer with the generated OpenAPI 3.1 specification

### 3.2.6 Documentation

The documentation module lists the generated SRS, SDD and README and renders the selected document; Figure 3.7 shows the SRS with its introduction, functional and non-functional requirements.

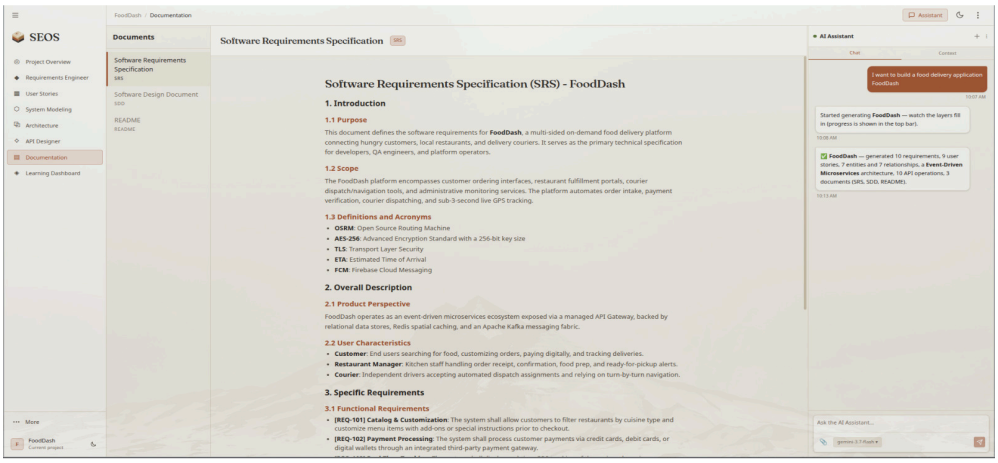



Figure 3.7: Generated Software Requirements Specification for FoodDash

### 3.2.7 Learning Dashboard

The Learning Dashboard tracks progress across six modules, displays a knowledge graph of related concepts and recommends further reading (Figure 3.8).

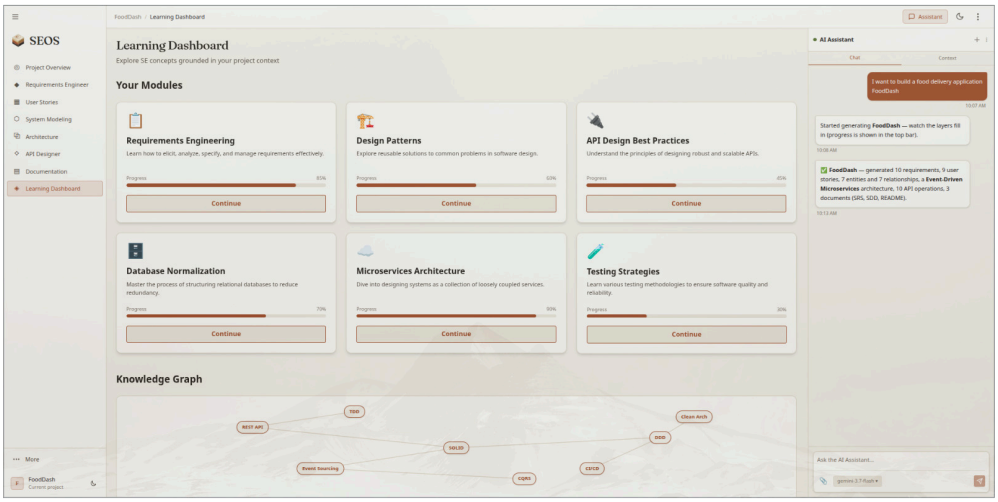



Figure 3.8: Learning Dashboard with module progress, knowledge graph and recommended reading

## 3.3 Performance Metrics

At mid-semester the prototype implements the full workspace shell and the first four pipeline stages for the FoodDash case study, as summarised in Table 3.1.

| Module | Output for FoodDash | Count | Status |
|---|---|---|---|
| Requirements Engineer | FRs / NFRs with priority (2 approved) | 6 | Done |
| User Story Generator | Stories with acceptance criteria (2 approved) | 4 | Done |
| UML / ER Designer | Editable ER model, 6 entities | 1 | Done |
| Architecture Designer | Event-driven microservices, 8 components | 1 | In progress |
| API Designer | OpenAPI 3.1 spec (users, restaurants, orders) | 1 | Prototype |
| Documentation Engine | SRS, SDD, README | 3 | Prototype |
| Learning Dashboard | 6 modules, knowledge graph, reading list | 6 | Prototype |
| RAG Layer | Knowledge base and retrieval | – | In progress |

Table 3.1: Module Implementation Status at Mid-Semester

The success criteria against which the final system will be evaluated are listed in Table 3.2.

| Metric | Target | Current |
|---|---|---|
| End-to-end workflow | One-paragraph idea reaches downloadable docs | Partial (4/6 stages) |
| Model synchronisation | Diagram edits update model and downstream output | Edit UI built |
| API generation | OpenAPI valid and importable without fixes | Spec generated |
| AI explanations | Relevant, grounded, understandable | RAG in progress |
| User evaluation | $\geq 5$ external users complete workflow | Planned (Month 5) |
| Deployment | Publicly accessible demo | Planned (Month 6) |

Table 3.2: Success Metrics and Current Status

## 3.4 Analysis

Walking through the FoodDash case study surfaced both strengths and concrete defects that guide the next phase:

- **Pipeline Continuity:** A single idea produced connected requirements, stories, an ER model, an architecture and an API specification within one workspace, confirming the value of the shared Project Model over separate tools.
- **Cross-Artefact Inconsistency Detected:** The API Designer's endpoint list shows `/api/v1/users` operations while the generated YAML begins with `/restaurants`; the two views are not yet derived from the same model snapshot. This is exactly the class of defect the trace link and `model_version` mechanism (Eq. 2.2) is designed to eliminate.
- **Model Completeness:** In the entity editor, PK/FK flags for generated entities are left unset, and requirement identifiers render as a truncated `REQ-00` prefix. Both are being fixed by enforcing these fields in the generation schema.
- **Architecture Scope:** The recommended polyglot stack (Go, Node.js, Python) is justified for a production system but may be heavy for a student project; the engine will be tuned to weigh team size and skill constraints when ranking styles.

# Chapter 4
# Conclusion

## 4.1 Summary of Work

SEOS has been designed and partially implemented as an AI-powered Software Engineering Operating System that carries a software idea through requirements, user stories, UML/ER modeling, architecture, API design and documentation inside one workspace. The prototype demonstrates the full workspace, a working requirements and user-story pipeline, an interactive ER canvas with entity editing, an architecture recommendation with trade-offs, an OpenAPI generator, document generation and a Learning Dashboard, all on the FoodDash case study.

## 4.2 Key Findings

- **A Shared Model Matters More Than Generation:** The most valuable property of SEOS is not that it generates artefacts, but that they are stored as linked objects. The inconsistencies found in the prototype appear exactly where a view is not yet derived from the model.
- **Structured Output Makes AI Reliable Enough to Persist:** Requiring schema-valid JSON with a bounded repair loop, followed by human approval, keeps malformed or contradictory LLM output out of the Project Model.
- **Explanations Turn a Tool into a Learning Environment:** Pairing each decision (e.g. choosing event-driven microservices) with its trade-offs and grounded references gives learners the "why", not just the "what".

## 4.3 Future Work and Recommendations

The remaining roadmap (Months 4–6) and documented extensions are:

1. **Two-Way Diagram Synchronisation:** Complete the patch-based model update and stale-artefact propagation (Figure 2.6), with a one-way fallback if full

synchronisation proves too costly.

2. **RAG Integration and OpenAPI Validation:** Finish the curated knowledge base and citation display, and validate every generated OpenAPI document automatically in CI.
3. **Usability Study and Deployment:** Run the workflow with at least five external users, then deploy on Vercel and Render with Docker and GitHub Actions.
4. **Out of Scope (Future Extensions):** AI code review, automated test-case generation, sprint/Kanban planning and a full coding assistant are deliberately excluded to keep the core pipeline deep rather than broad.

## 4.4 Final Remarks

SEOS shows that combining a connected engineering model, validated AI generation and grounded explanations can keep software artefacts consistent while helping learners understand how and why software should be engineered. In the team's words: *we learn software engineering by building a software engineering operating system that helps others learn and apply software engineering.*

# Bibliography


[Fas24] S. Ramírez. *FastAPI Documentation*. 2024. URL: `https://fastapi.tiangolo.com` (cit. on p. 6).

[IEEE98] IEEE. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. IEEE Computer Society, 1998. (cit. on p. 11).

[JBS15] Michael B. Jones, John Bradley, and Nat Sakimura. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force (IETF), 2015. (cit. on p. 6).

[LC24] LangChain Inc. *LangChain Documentation*. 2024. URL: `https://python.langchain.com` (cit. on p. 6).

[Lew20] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020. (cit. on p. 4).

[Met24] Meta Platforms Inc. *React Documentation*. 2024. URL: `https://react.dev` (cit. on p. 6).

[OAS21] OpenAPI Initiative. *OpenAPI Specification v3.1.0*. 2021. URL: `https://spec.openapis.org/oas/v3.1.0` (cit. on p. 11).

[PGV24] pgvector contributors. *pgvector: Open-source Vector Similarity Search for Postgres*. 2024. URL: `https://github.com/pgvector/pgvector` (cit. on p. 6).

[PM19] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th. New York, NY: McGraw-Hill Education, 2019. (cit. on p. 3).

[RF24] xyflow. *React Flow Documentation*. 2024. URL: `https://reactflow.dev` (cit. on p. 6).

[Som15] Ian Sommerville. *Software Engineering*. 10th. Boston, MA: Pearson, 2015. (cit. on p. 3).